

Práctica 1: Aprendizaje no supervisado

La paleta de los pintores

Machine Learning

Máster Universitario en Inteligencia Artificial

Curso académico 2026/2027

Antonio M. Durán Rosal

Práctica 1: la paleta de los pintores

Esta práctica trata de **aprendizaje no supervisado**: encontrar estructura en datos que no traen una respuesta con la que comprobarla. Los datos son **entre 48 y 59 cuadros** de dominio público de la colección del Art Institute of Chicago, y **cada grupo tiene los suyos**. Primero diseñaréis y programaréis vuestra propia **variante de K-means** como estimador de scikit-learn y la usaréis para extraer la paleta de un cuadro; después describiréis cada cuadro por sus colores, agruparéis los cuadros sin saber quién los pintó y, solo al final, compararéis los grupos con los autores. La última parte usa PCA para dibujar un mapa de los cuadros.

El objetivo no es solo usar las bibliotecas, sino **interpretar** lo que devuelven y **justificar** cada decisión. La evaluación pondera el código y el razonamiento escrito: un resultado sin comentar puntúa muy poco. Usa los contenidos de los **Temas 2 y 3** y no depende de ninguna otra práctica ni del boletín de problemas.

Grupos y calendario

- **Grupos de cuatro**, los publicados por el profesor. Cada grupo recibe su **propio conjunto de cuadros**: seis autores distintos para cada grupo, siempre con uno que pinta sobre papel o en laca. Los resultados de dos grupos no son comparables entre sí, y no tienen por qué serlo.
- **Martes 27 de octubre de 2026**: presentación de la práctica al final de la clase. Desde ese momento cada grupo trabaja de forma autónoma.
- **Martes 3 de noviembre de 2026**: sesión de trabajo en clase de dos horas. Conviene llegar con la variante de K-means elegida y, a ser posible, programada.
- **Entrega: lunes 9 de noviembre de 2026, antes del comienzo de la clase**, en la que empieza el tema siguiente.

Puntuación

Hay **cinco ejercicios** y **no** pesan lo mismo. El total es **10.0**.

Ejercicio	Contenido	Puntos
1	Vuestra variante de K-means y la paleta de un cuadro	3.0
2	De cuadros a vectores: un diccionario común de colores	1.5
3	K-means y vuestra variante sobre los cuadros	2.0
4	Agrupamiento jerárquico de los cuadros	2.0
5	PCA: el mapa de los cuadros	1.5
	Total	**10.0**

La puntuación de cada apartado aparece al final de cada ejercicio.

Notación, fija para toda la práctica

- n es el número de observaciones y k el número de características. **La k minúscula nunca es otra cosa**: en particular, nunca es un número de grupos ni de colores.
- K es un número en mayúscula que **se define en cada ejercicio**: el número de colores de la paleta en el Ejercicio 1 y el número de grupos de cuadros en los Ejercicios 3 y 4.
- K_{dic} es el tamaño del diccionario común de colores del Ejercicio 2, fijado por el curso en $K_{\text{dic}} = 24$, y r el número de componentes principales.
- J es el objetivo de K-means, la suma de distancias al cuadrado de cada observación al centro de su grupo:

$$J = \sum_{c=1}^{K} \sum_{\mathbf{x} \in C_c} \|\mathbf{x} - \boldsymbol{\mu}_c\|^2.$$

  Si vuestra variante minimiza otro objetivo, definidlo con la misma claridad.
- Las decisiones de escala se **declaran y se nombran**: **escalado min–max** lleva cada característica a $[0, 1]$; **estandarización** resta la media y divide por la desviación típica.

Datos, y dónde están

Todo lo que necesita la práctica está **incluido** en el paquete de vuestro grupo. **Ningún ejercicio necesita conexión a la red** y ninguno puede descargar nada.

Datos	Ubicación
Los cuadros de vuestro grupo (JPEG, lado mayor de 512 píxeles)	`Datos/cuadros/`
Tabla de cuadros: `row_id`, fichero, ancho, alto	`Datos/cuadros.csv`
Número de grupo y los tres cuadros del Ejercicio 1	`Datos/grupo.json`
Metadatos: autor, época, año, técnica, título *(solo en la revelación)*	`Datos/metadatos.csv`
Procedencia y licencia	`Datos/PROVENANCE.md`

Los nombres de fichero y el `row_id` se asignaron en un orden aleatorio, de modo que no revelan el autor. **Alinead siempre por `row_id`.**

Aleatoriedad: las tres semillas del curso

Comparar inicializaciones forma parte del Ejercicio 3, así que la práctica **no** usa una única semilla. Usa **exactamente estas tres**, y ninguna otra:

```
P1_SEEDS = [13, 41, 97]
```

Donde haga falta una sola semilla se usa **la primera**, `SEED = P1_SEEDS[0] = 13`. **No las sustituyáis por otras.**

El sello no supervisado: leed esto antes del Ejercicio 3

Los cuadros tienen autor, pero **solo podéis usar los metadatos después de congelar *las dos* decisiones de agrupamiento**. El sello cubre los Ejercicios 3 y 4: son dos elecciones de modelo sobre los mismos datos, y congelar una con la otra abierta dejaría que los autores se filtraran en la segunda.

El orden de ejecución del cuaderno está fijado y se evalúa (en los encabezados, el signo entre las dos partes es una raya, «—», no un guion):

1. Ejercicios 1 y 2, y los apartados (a) y (b) del Ejercicio 3;
2. una celda Markdown con el encabezado exacto
   DECISIÓN CONGELADA — K-MEANS;
3. los apartados (a) a (c) del Ejercicio 4;
4. una celda Markdown con el encabezado exacto
   DECISIÓN CONGELADA — AGLOMERATIVO;

5. solo entonces, la etapa común
   REVELACIÓN — VALIDACIÓN EXTERNA, con los apartados 3(d), 4(d) y 4(e);
6. el Ejercicio 5.

Antes del paso 5, ninguna celda ejecutable puede cargar, inspeccionar ni derivar nada de `metadatos.csv`, ni siquiera para contar cuántos autores hay. La tabla `cuadros.csv` **no contiene ningún metadato**. Ver las miniaturas está permitido, porque son los datos, pero vuestros argumentos no pueden apoyarse en reconocer a un pintor.

Se comprueba de forma automática. El cuaderno incluye una función `check_seal` que lee el JSON del propio cuaderno y verifica que los dos encabezados existen **exactamente una vez cada uno**, que la congelación de K-means va primero y que el primer acceso **ejecutable** a los metadatos ocurre después de la segunda congelación. Ejecutadla sobre el fichero guardado antes de entregar. Para comparar *dos particiones vuestras* entre sí antes de la revelación usad la función proporcionada `rand_ajustado`: `adjusted_rand_score` queda reservado a la revelación.

Ningún K, algoritmo ni enlace se puede elegir maximizando un índice externo. Probar varias configuraciones, ver cuál se parece más a los autores y presentarla como un resultado no supervisado es exactamente el error que esta secuencia evita. Es una **penalización automática**: cuesta la puntuación de los apartados afectados aunque todos los números sean correctos.

Qué se entrega

- **Un cuaderno**, `p1_grupo<N>.ipynb` con vuestro número de grupo, ejecutado **de arriba abajo en una sola pasada**, con las salidas guardadas y redactado en **español**. Partid del cuaderno proporcionado `Cuaderno/p1_alumno.ipynb`. Varios ficheros en lugar de uno se penalizan.
- `USO_IA.md`. El uso de IA generativa está **permitido con declaración**: indicad la herramienta, el propósito, las secciones afectadas y cómo comprobasteis el resultado. Si no la usasteis, basta una frase. Un fichero ausente o vacío resta puntos, y el grupo es plenamente responsable de todo lo entregado.

La interpretación va en celdas Markdown, no en comentarios de Python. Un número correcto sin una lectura escrita obtiene la puntuación de cálculo y ninguna de la de interpretación.

1. Vuestra variante de K-means y la paleta de un cuadro (3.0)

Cada píxel de un cuadro es una observación con $k = 3$ características: sus intensidades de rojo, verde y azul, que la función proporcionada `a_pixeles` lleva a $[0, 1]$. Si se agrupan los píxeles y se sustituye cada uno por el centro de su grupo, el cuadro queda pintado con una **paleta de K colores**. Aquí K **es el número de colores de la paleta**.

En este ejercicio **cada grupo diseña y programa una variante de K-means que no esté en scikit-learn**, y la escribe como un estimador más de la biblioteca. Parte del trabajo es **investigar** cómo se construye un estimador propio de scikit-learn. La elección de la variante es libre; algunas ideas, por si sirven de punto de partida:

- **una inicialización determinista**: por máxima separación, a partir de las componentes principales, *global K-means*...;
- **otra distancia u otro representante**: K-medianas (distancia ℓ_1), K-medoides, K-means esférico (coseno);
- **otro objetivo**: K-means recortado (robusto a atípicos), con pesos por característica, con restricción de tamaño de los grupos, difuso (*fuzzy c-means*);
- **una variante pensada para histogramas**, que os servirá en el Ejercicio 3: K-means con la divergencia de Kullback–Leibler;
- **una variante que elija** K, como *X-means*.

No vale un algoritmo que ya esté en scikit-learn (`KMeans`, `MiniBatchKMeans`, `BisectingKMeans`...), ni cambiar solo un parámetro de `KMeans`, ni ajustar llamando a `KMeans` por dentro.

a) **Diseño.** Qué variante habéis elegido; en qué se diferencia del K-means de Lloyd (inicialización, distancia, actualización de los centros, objetivo, criterio de parada...); por qué no está en scikit-learn; de dónde sale (una referencia, si existe); y qué esperáis que mejore o empeore.

b) **Implementación como estimador de scikit-learn** (scikit-learn 1.6 o posterior). Una clase que:

   - herede de `ClusterMixin` y de `BaseEstimator`, en ese orden; también de `TransformerMixin`, si tiene sentido;
   - guarde en `__init__` los parámetros tal cual, sin lógica;

- tenga `fit(X, y=None)` que devuelva `self`, y `predict`;
- exponga lo aprendido con guion bajo final: `labels_`, `cluster_centers_` (si tiene sentido), el valor del objetivo (por ejemplo `inertia_`) y `n_iter_`;
- tenga `random_state` si la variante es aleatoria;
- no recorra las observaciones con bucles de Python: los cuadros tienen hasta $260\,000$ píxeles.

Debe pasar la función proporcionada `comprobar_estimador`. Investigad también `sklearn.utils.estimator_checks.check_estimator`.

c) **Verificación.** Demostrad que vuestra implementación hace lo que decís. Si la variante tiene un caso particular que coincide con `KMeans` (por ejemplo, unos centros iniciales dados), comparad las dos desde ese caso: el objetivo (diferencia relativa) y las particiones (con `rand_ajustado`). Si no lo tiene, diseñad otra comprobación: un ejemplo pequeño resuelto a mano, una propiedad que deba cumplirse... Dibujad la evolución del objetivo a lo largo de las iteraciones y explicad su forma.

d) **La paleta de tres cuadros.** Para cada cuadro de `CUADROS_EJ1` y para $K \in \{3, 6, 12\}$, ajustad **vuestra variante** sobre **todos** los píxeles, sustituid cada píxel por el centro (o el representante) de su grupo y mostrad el original y las tres versiones, con su paleta debajo (`mostrar_paleta`). Ajustad también `KMeans(n_clusters=K, n_init=1, random_state=SEED)` y comparad en una tabla el objetivo y el tiempo de ajuste de los dos. Responded:

- ¿Qué se conserva y qué se pierde al bajar K? ¿A partir de qué K deja cada cuadro de parecer claramente dañado?
- ¿Por qué los colores de la paleta parecen más «apagados» que los del cuadro?
- ¿Qué les pasa a los colores llamativos que ocupan poca superficie? Explicadlo a partir de lo que minimiza el objetivo.
- ¿Qué hace vuestra variante distinto de `KMeans`, y se cumple lo que esperabais en (a)? Si no se cumple, decidlo: es un resultado tan válido como el contrario.

Puntuación: (a) 0.5; (b) 1.0; (c) 0.5; (d) 1.0. *Una «variante» que ya está en scikit-learn, que solo cambia un parámetro de `KMeans` o que ajusta llamando a `KMeans` anula (a), (b) y (d).*

2. De cuadros a vectores: un diccionario común de colores (1.5)

Para agrupar **cuadros** hace falta describirlos a todos con el mismo vector. La idea es construir un **diccionario** de $K_{\text{dic}} = 24$ colores común a todos vuestros cuadros y describir cada cuadro por la proporción de su superficie que ocupa cada color. El diccionario se construye con `sklearn.cluster.KMeans`.

a) **Por qué no se comparan paletas directamente.** Explicad en dos o tres frases por qué no se pueden comparar posición a posición las paletas de dos cuadros obtenidas por separado.

b) **El diccionario.** Tomad `PIX_POR_CUADRO` $= 2000$ píxeles de cada cuadro, sin reemplazo, con un único `rng = np.random.default_rng(SEED)` y recorriendo los cuadros en el orden de `cuadros.csv`. Ajustad `KMeans(n_clusters=K_DIC, n_init=1, random_state=SEED)` sobre todos esos píxeles juntos y mostrad los 24 colores. **Usad exactamente estos parámetros**.

c) **La tabla de histogramas.** Asignad **todos** los píxeles de cada cuadro a su color más cercano del diccionario y construid la tabla H, de $n \times K_{\text{dic}}$, con la proporción de cada color en cada cuadro. Comprobad que cada fila suma 1 y dibujad la fila de dos cuadros que elijáis como un diagrama de barras con los colores del diccionario.

d) **Qué describe H, y la escala.** ¿Qué describe una fila de H y qué ignora? ¿Vais a escalar H antes de agrupar? Declarad la decisión, nombradla y argumentadla: las dos respuestas son defendibles, pero solo una declarada y argumentada puntúa.

Puntuación: (a) 0.3; (b) 0.4; (c) 0.4; (d) 0.4.

3. K-means y vuestra variante sobre los cuadros (2.0)

La entrada es la tabla H del Ejercicio 2, con la decisión de escala que declarasteis. Aquí K **es el número de grupos de cuadros**.

a) **La rejilla.** Para $K \in \{3, 4, 5, 6, 7, 8\}$, ajustad `KMeans` una vez con **cada** semilla de `P1_SEEDS` y con `n_init=1`, y **vuestra variante** (con las tres semillas si es aleatoria, una vez si es determinista). Para cada ejecución, informad de J, de la silueta y de los tamaños de grupo; para `KMeans`, además, de la **media** de J en las tres semillas y de su dispersión. **Informad de la media, no del mejor de los tres**: el menor J de tres reinicios habla de vuestra inicialización más afortunada, no de K.

b) **Las reglas, y después una partición.** Este apartado necesita **tres** decisiones, y mezclarlas es el error que hay que evitar:

- **¿Qué K?** A partir de la evidencia **no supervisada**. J baja con K por construcción, así que «J **es mínimo en** $K = 8$» **no es un argumento**: usad la forma

de la curva, la silueta, los tamaños de grupo, la dispersión entre semillas o cualquier otro criterio que podáis defender.

- **¿Qué algoritmo?** `KMeans` o vuestra variante, con una regla no supervisada declarada.
- **¿Qué ejecución? Solo después** de fijar K y algoritmo, si hay varias; por ejemplo, la de menor J **con el** K **ya fijado**.

Escribid las reglas antes de aplicarlas. Después mostrad la rejilla de miniaturas de esa partición (`rejilla_miniaturas`) y describid cada grupo **en términos de color y de luz**.

c) **Congelación.** Escribid la configuración que se mantiene (K, algoritmo, semilla si procede, escalado y reglas), con su argumento **no supervisado**, en una celda Markdown con el encabezado exacto DECISIÓN CONGELADA — K-MEANS. Lo que se congela es **una** partición. Si parte de vuestra evidencia apunta en otra dirección, **decidlo**: una elección congelada con una limitación reconocida puntúa mejor que una presentada como indiscutible.

El apartado (d) se hace en la revelación, después de congelar también el Ejercicio 4:

d) **Validación externa.** Calculad el índice de Rand ajustado (ARI) y la información mutua ajustada (AMI) de la partición congelada **y de ninguna otra**, frente al **autor** y frente a la **época**. Construid la tabla de contingencia grupo $\times$ autor y volved a mostrar la rejilla, ahora con el autor de cada cuadro a la vista. ¿Qué significaría un valor cercano a 0, y uno cercano a 1? Y la pregunta de fondo: **¿qué ha agrupado realmente el algoritmo?** ¿Autores, épocas, la luz, el soporte, una serie de cuadros?

Puntuación: (a) 0.5; (b) 0.5; (c) 0.3; (d) 0.7. *Elegir K o algoritmo maximizando un índice externo anula (c) y (d).*

4. Agrupamiento jerárquico de los cuadros (2.0)

La misma tabla H y el mismo sello: **los apartados (a) a (c) no usan metadatos**. Aquí K es también el número de grupos.

a) **Los cuatro enlaces.** Ajustad el agrupamiento aglomerativo con los cuatro criterios de enlace (*linkage*) que ofrece scikit-learn: `ward`, `complete`, `average` y `single`. Indicad la restricción de métrica: `ward` solo admite la euclídea, mientras que los otros tres aceptan `euclidean`, `l1`, `l2`, `manhattan`, `cosine` y `precomputed`. Mostrad qué ocurre si se le pide a `ward` otra métrica. **Es una frase sobre la API,**

no una invitación a estudiar métricas: todos los ajustes de este ejercicio usan la euclídea.

b) **Dos criterios, declarados antes de aplicarlos.**

- **Dentro de cada enlace, ¿qué K?** Cada enlace es una jerarquía distinta y los saltos entre alturas de fusión caen en sitios distintos: **cada enlace tiene su propio K**.
- **Entre enlaces, ¿cuál preferís?** Una regla que pueda elegir uno, **solo con evidencia no supervisada**. «El que se ve mejor» no es una regla.

Una misma familia de reglas puede servir para las dos cosas; por ejemplo, cortar por debajo del mayor salto relativo entre alturas de fusión consecutivas y preferir el enlace cuyo salto seleccionado sea mayor.

c) **Un dendrograma.** Dibujad el dendrograma del enlace **preferido**, cortado en **su K**, con `dendrograma_con_miniaturas`. Leedlo: ¿qué fusiones son tempranas y baratas, y cuál es tan cara que vuestra regla cortó por debajo?

Antes de la revelación, escribid en una celda Markdown con el encabezado exacto DECISIÓN CONGELADA — AGLOMERATIVO **dos** objetos: **un K por enlace** y **un enlace preferido**. Registrad también los tamaños de grupo observados en cada enlace con su K: el apartado (e) los necesita y deben existir antes de leer ningún metadato.

Los apartados (d) y (e) se hacen en la revelación:

d) **Validación externa de cada enlace.** Los mismos índices que en 3(d) para **cada enlace con su K congelado**; indicad el K en cada fila y marcad el preferido. Después, la comparación: el enlace con **mejor** acuerdo con el autor y el que preferisteis **sin** metadatos, ¿coinciden? Si no, **decidlo sin rodeos**: sin etiquetas no había forma de saberlo, y es un resultado más instructivo que la coincidencia. **No editéis ninguno de los dos objetos congelados para que coincidan.**

e) **Los cuatro enlaces, comparados.** Con los tamaños que registrasteis, comparad lo que hicieron los cuatro enlaces con estos datos y explicad cada uno a partir de lo que mide entre dos grupos. `single` mide la distancia entre los dos puntos **más cercanos**, así que un cuadro intermedio puede unir dos zonas lejanas: es el **encadenamiento**. **Comprobad si vuestra ejecución lo muestra**; si no lo muestra, decidlo.

Puntuación: (a) 0.3; (b) 0.4; (c) 0.4; (d) 0.5; (e) 0.4.

5. PCA: el mapa de los cuadros (1.5)

PCA sobre la tabla H, centrada y con la misma decisión de escala del Ejercicio 2(d), para **ver** los cuadros.

a) **Varianza explicada.** Informad de la razón de varianza explicada de las primeras componentes, de la acumulada y de cuántas componentes hacen falta para el 90 %.

b) **El mapa y las cargas.** Dibujad los cuadros sobre las dos primeras componentes con `mapa_con_miniaturas`, y las **cargas** de CP1 y CP2 como un diagrama de barras con los colores del diccionario.

c) **Interpretación.** Describid CP1 y CP2 **con palabras de color**, a partir de las cargas. Después coloread el marco de cada miniatura por autor (ya revelado) y relacionad las componentes, los grupos de vuestra partición congelada y los autores.

Puntuación: (a) 0.4; (b) 0.5; (c) 0.6.

Las imágenes proceden de la colección del Art Institute of Chicago y se distribuyen bajo la designación Creative Commons Zero (CC0). La lista de obras de vuestro grupo está en `Datos/PROVENANCE.md`.